ACIBO JOHN CARL T.

JOANNA FLORES

SECTION-C

NOVEMBER 11, 2026

Java OOP laboratory exercise: ACIBO-FLORES-_CafeApp-System

1. Encapsulation

I used encapsulation in our Beverage class by making the Size variable private. I also used getters and setters, such as getSize() and setSize(), so I can safely access and update the beverage size.

2. Inheritance

I used inheritance in our Beverage class through extends MenuItem. This allows the Beverage class to reuse common information like the name, base price, and item code without creating them all over again.

3. Polymorphism

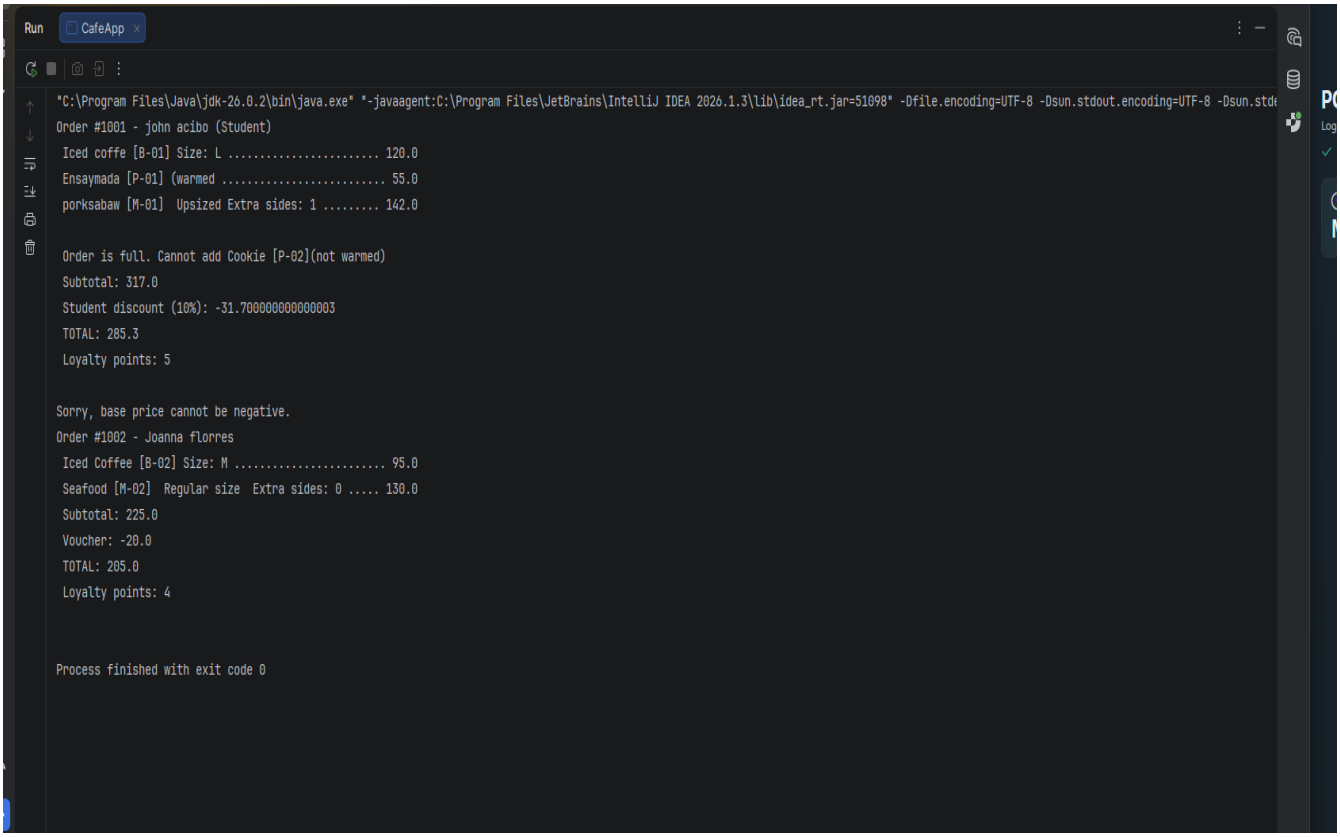
I used polymorphism in our Beverage class by overriding the computePrice() and getDescription() methods from the parent class. This allows beverages to have their own price calculation based on their selected size.

4. Abstraction

I applied abstraction through the MenuItem class, which contains common details and methods for our cafe products. This helps keep the code organized because each product class can focus on its own specific features.

5. Objects and Classes

I used classes to organize different parts of our cafe system, such as Customer, Beverage, Meal, Pastry, and Order. In CafeApp.java, I created objects from these classes to represent actual customers, products, and orders.



Section 6 – Challenges Encountered and How We Solved Them

1. Understanding Object-Oriented Programming

One of our challenges was understanding how to properly use classes, objects, inheritance, and encapsulation. We solved this by studying each concept and applying it step by step to our cafe system.

2. Managing Product Prices

We had difficulty calculating the correct prices, especially when adding extra charges for beverage sizes and meal options. We solved this by creating separate methods to calculate the prices based on the selected options.

3. Handling Customer Orders

Another challenge was managing customer orders and making sure the correct products and quantities were included. We solved this by using the Order class to organize the items and calculate the total amount.

4. Fixing Errors in the Code

We encountered errors while writing and testing our Java program. We solved them by checking our code carefully, identifying the mistakes, and testing the program again until it worked properly.

5. Applying Validation

We also faced challenges when handling invalid inputs, such as negative prices or adding items when the order was already full. We solved this by adding validation conditions to prevent incorrect data from being accepted.